

Revive Peptides Co: Whop tracking is not installed on the site

Prepared 7 September 2026 for the Revive Peptides Co development team. Business `biz_b3aIz02MxBmvIg` / `revivepeptidesco.com`. Everything below was measured against the live site and the live Whop event stream on 7 September 2026.

1. The finding

The Whop pixel has **never been installed** on `revivepeptidesco.com`. This is not a misconfiguration, a wrong event name, or a scope problem. There is no Whop code on the site at all, and the Whop business has never recorded a single pixel event.

The useful part: this is the **easy** version of this problem. The checkout is a normal WooCommerce checkout on your own domain, so both pieces below are ordinary WordPress work. No hosted checkout and no third party funnel builder is involved.

2. What was measured, and how

The site carries no Whop code

Checked in a real browser on three pages: the homepage, a product page (`/product/kpv/`), and the checkout page with a live cart loaded.

page	t.whop.tw scripts	window.whop	_wuid cookie	inline Whop code
<code>/</code>	none	undefined	none	none
<code>/product/kpv/</code>	none	undefined	none	none
<code>/checkout/</code> (cart loaded)	none	undefined	none	none

Zero network requests to `t.whop.tw` on any of them.

Why a browser and not curl. On two other stores in this group the Whop pixel is injected at runtime and never appears in the page source, so a source fetch reports it missing while it is plainly loading. A curl cannot answer this question; it can only produce a false absence.

The blocker was ruled out. The browser used has an extension that blocked `connect.facebook.net/en_US/fbevents.js` on this site. That alone could have suppressed a Whop request and manufactured this whole finding, so it was tested directly: in the same browser and the same session, a sister store loads `t.whop.tw/s.js` and `t.whop.tw/sc.js`, `window.whop.track` is a live function, and the `_wuid` cookie is set. The blocker does not touch Whop. The absence here is real.

The Whop business has never recorded a pixel event

`GET /api/v1/events` for this business over a 29 day window returns **9 events, every one of them an account setup event from the day the business was created** (3 September 2026): affiliate created, business updated twice, invite created, invite accepted, and four dashboard recommended-action rows.

There is **not one** `pixel.custom` row. No `view_content`, no `add_to_cart`, no `initiate_checkout`, no `whop_purchase`. The business also currently has **0 ad campaigns**.

Read carefully: on `GET /events` every custom event, purchases included, carries `event_name`: `"pixel.custom"`, and the real name lives in `custom_name`. Filtering on `event_name` for "purchase" returns 0 even on an account that provably has one. The tally above was taken across both fields, so this zero is a real zero and not that artifact.

3. The funnel, as it actually is

WordPress and WooCommerce, behind Cloudflare. Also present: GA4 (four gtag containers), the Meta pixel, Klaviyo, and a Tagada tracker.

The checkout is on your own domain. This was tested rather than assumed:

- `/checkout/` renders the native WooCommerce billing form with a **Place order button on the page itself**.
- There is **no payment iframe**. The only frames present are a BBB trust seal image and a 1px Stripe helper frame.
- **checkout.revivepeptidesco.com does not exist** (404). There is no checkout subdomain.
- The WooCommerce `order-received` endpoint is registered on-domain and responds at `/checkout/order-received/`.

Three payment methods are offered: **Zelle** (advertised at 3 percent off), **Same-Day Bank Payment**, and **card through the Tagada gateway**.

Two store rules that shape the funnel: a **\$75.00 minimum order** is enforced, and US shipping is free over \$200.

4. What needs to happen

4a. Load the pixel site wide

Add this to the site header so it runs on every page, including checkout and the order confirmation page. It is already scoped to the correct business id.

```
<script>
!function(w,d,s,u,n,a,b){if(w[n])return;a=w[n]={q:[],t:new Date,s:[],o:u,track:function(){
a.q.push([+new Date].concat([].slice.call(arguments)))},setScope:function(){a.s=
[].slice.call(arguments).filter(function(x){return typeof x==="string"});a.q.push([+new
Date,"setScope"].concat(a.s))},scope:function(){var c=
[].slice.call(arguments);return{track:function(){a.q.push([+new
Date].concat([].slice.call(arguments)).concat([__scope:c])}}}};b=d.createElement(s);b.async
c=1;b.src=u+"/s.js";d.getElementsByTagName(s)[0].parentNode.insertBefore(b,d.getElementsByTagName(s)[0])}
(window,document,"script","https://t.whop.tw","whop");
whop.setScope("biz_b3aIz02MxBmvIg");
whop.track("page");
</script>
```

Paste it exactly as it is. **Do not replace the loader body with a comment or a placeholder.** A stubbed body still parses cleanly, so it looks installed, but it never creates `window.whop`. The very next line then throws `whop is not defined`, nothing is ever tracked, and the thrown error can take out other scripts on the same page.

4b. Fire the purchase on the order confirmation page

Because the checkout is on-domain, the confirmation page is WooCommerce's own `/checkout/order-received/<id>/`, and the clean place to hook it is `woocommerce_thankyou`.

```
add_action( 'woocommerce_thankyou', function ( $order_id ) {
    if ( ! $order_id ) { return; }
    $order = wc_get_order( $order_id );
    if ( ! $order ) { return; }

    // only fire for an order that is actually paid
    if ( ! $order->is_paid() ) { return; }

    // fire once per order, ever
    if ( $order->get_meta( '_whop_purchase_sent' ) ) { return; }
    $order->update_meta_data( '_whop_purchase_sent', 1 );
    $order->save();

    $value    = (float) $order->get_total();
    $currency = strtolower( $order->get_currency() );
    ?>
    <script>
    whop.track("whop_purchase", {
        event_id: <?php echo wp_json_encode( (string) $order_id ); ?>,
        value:    <?php echo wp_json_encode( $value ); ?>,
        currency: <?php echo wp_json_encode( $currency ); ?>
    });
    </script>
    <?php
}, 10, 1 );
```

The event name goes in the FIRST ARGUMENT. This is the one that fails silently. Write `whop.track("whop_purchase", {...})`. Do **not** write `whop.track("purchase", { event_name: "whop_purchase", ... })`. In the shipped pixel at `t.whop.tw/s.js` the conversion builder reads only `name`, `event_id`, `value`, `currency`, `source`, `optimization_value`, `email` and `phone` off the second argument, so an `event_name` property is **silently dropped**. The wrong form files the conversion under `purchase`, campaigns optimizing on `whop_purchase` read zero forever, and Whop has **no delete endpoint**, so every one of those rows is permanent. The only other correct spelling is `whop.track("custom", { name: "whop_purchase", ... })`, because `name` is read only when the first argument is literally `"custom"`.

This rule is **inverted** on Whop's server side API (`POST /api/v1/events`), where the custom name goes in `event_name` and a `custom_name` you send is ignored. Knowing one surface gives you the wrong answer on the other. This section is the browser pixel.

5. Things that fail quietly on this specific store

1. **Zelle and bank-transfer orders may never render a paid confirmation page in the buyer's browser.** Those can be marked paid later, out of band, by a human or a webhook, and a browser pixel on the thank-you page cannot see that. The `is_paid()` guard above is deliberately strict, so such orders will be **missing** rather than wrong. If Zelle is a meaningful share of revenue, close that gap server side against `POST /api/v1/events` instead, where the name goes in `event_name`. Decide this before judging the install by whether the numbers reconcile.
2. **setScope replaces the scope, it does not add to it.** If anything else on the page calls `setScope` after you do, the business id is gone and events go elsewhere. Order matters.
3. **A 200 response proves acceptance, never correctness.** Read the event back out of the API and check the company id, the name and the value.
4. **event_id is the dedup key and there is no delete endpoint.** A wrong or duplicated `event_id` is permanent, so use the real order id and test with a throwaway value first.
5. **currency must be lowercase, and value: 0 is stored as null**, so a compounded or zero-total order reads as a missing value rather than a zero.
6. **The \$75 order minimum sits in front of every purchase.** Nothing to fix in the tracking, but when you compare add-to-cart counts against purchases, remember that carts under \$75 are refused at checkout by design.

6. How to verify it worked

Do not rely on the Whop dashboard tile.

1. On any page, confirm in the network tab that `t.whop.tw/s.js` loads and that `window.whop.track` is a function. **Check this with ad blocking off**, or you will chase a block as if it were a bug.
2. Place one real test order through the **card** path and, on the confirmation page, confirm a `POST` to `t.whop.tw/conversions` whose **request body carries biz_b3aIz02MxBmvIg**.
3. Read it back and confirm it appears **exactly once**, with `custom_name` equal to `whop_purchase`, the right value, and lowercase currency:

```
GET https://api.whop.com/api/v1/events?company_id=biz_b3aIz02MxBmvIg&from=<iso>&to=<iso>
```

`from` and `to` are both required, and the range **cannot exceed 30 days** or the call returns 400. That 400 is a payload error, not an auth error.

If step 2 passes and step 3 returns nothing, the pixel is firing into a different business, which is failure mode 2 above.

7. What is not claimed here

The order confirmation page was **not** observed directly, because reaching it requires a completed payment, and no order was placed on this store. That one page is therefore measured by inference rather than directly.

The inference is still firm: the pixel is absent on the homepage, on a product page, and on the checkout page itself, and the Whop business holds zero pixel events of any kind across 29 days. There is no version

of "installed, but only on the thank-you page" that produces that.